

GammaTon 플러그인 — 코드 구조 & 아키텍처 가이드

논문: "**Visual Simulation of Weathering by γ -ton Tracing**" (Chen et al., SIGGRAPH 2005)

작성일: 2026-05-24

목차

1. 패널 열기
2. 전체 레이어 구조
3. 파일별 역할
4. 핵심 데이터 구조
5. Run 버튼 전체 실행 플로우
6. γ -ton 1개 트레이싱 코드 경로
7. 이터레이션 루프 상세
8. 텍스처 파이프라인
9. 코드 수정 가이드

1. 패널 열기

UE 에디터 상단 메뉴:

Tools → Weathering Simulation → GammaTon Dust Simulator

GammaTon.cpp의 RegisterMenus()가 에디터 시작 시 이 메뉴 항목을 등록합니다.

클릭 시 OpenPanel()이 460×700 고정 크기 Slate 창으로 SGammaTonPanel을 엽니다.

2. 전체 레이어 구조

UE Editor Layer

SGammaTonPanel (Slate UI)

- ├─ 파라미터 읽기/쓰기
- ├─ OnRunClicked() — 전체 시뮬레이션 실행
- ├─ OnReapplyClicked() — 텍스처 재적용만
- └─ OnTraceRayClicked() — 단일 레이 디버그

GammaTonMeshBridge

UStaticMesh → GTMesh

UE → 순수 C++ 변환

GammaTonTextureBridge

GTObjTexture → UTexture2D

순수 C++ → UE 에셋 변환

GammaTonSourceVisualizer / GammaTonRayVisualizer
(뷰포트 디버그 렌더링)

Simulation Core Layer (UE 의존성 없음 - 순수 C++)

GTSimulator

- └─ runIteration() γ-ton 추적 1회
- └─ traceTon() γ-ton 1개 경로 처리
- └─ applyCrossChannel() 채널 간 재질 상호작용
- └─ updateReflectance() 표면 반사율 갱신
- └─ traceTonDebug() 디버그용 경로 기록

GTRayIntersector (Intel Embree4 래퍼)

- └─ addMesh() / commit() BVH 빌드
- └─ intersect() 레이-삼각형 교차 테스트

GTCore.h (데이터 타입 전용)

GTVec3, GTMesh, GTSurfel, GTObjTexture, GTTonType, ...

↕ (addMesh 시 GTMesh 전달)

ThirdParty: Intel Embree 4

고성능 BVH 레이트레이싱 라이브러리

GTRayIntersect.cpp에서 embree4.dll을 통해 호출

핵심 설계 원칙: 시뮬레이션 코어(Core/ 폴더)는 UE 타입을 전혀 사용하지 않습니다.

UE ↔ 코어 간 변환은 **MeshBridge**와 **TextureBridge**가 담당합니다.

이 구조 덕분에 코어 로직을 UE 없이 단독 테스트할 수 있습니다.

3. 파일별 역할

Core (순수 C++, UE 없음)

파일	역할
Core/GTCore.h	모든 데이터 구조 정의. 이 파일 하나로 전체 데이터 모델을 파악할 수 있음
Core/GTSimulator.h/.cpp	γ-ton 트레이싱 엔진. 시뮬레이션 물리 전부 여기에 있음
Core/GTRayIntersect.h/.cpp	Intel Embree를 래핑하는 레이캐스터. intersect() 한 함수가 핵심

UE Bridge (UE ↔ Core 변환)

파일	역할
GammaTonMeshBridge.h/.cpp	UStaticMeshComponent → GTMesh + GTSurfel 변환, Embree BVH 빌드
GammaTonTextureBridge.h/.cpp	GTObjTexture → UTexture2D 저장 (/Game/GammaTon/), 머티리얼 MID 주입

UI & 진입점

파일	역할
<code>GammaTon.h/.cpp</code>	UE 모듈 진입점. 메뉴 등록 및 패널 창 생성
<code>SGammaTonPanel.h/.cpp</code>	Slate UI 전체. 파라미터 바인딩, Run/Reapply/TraceRay 핸들러

뷰포트 디버그

파일	역할
<code>GammaTonSourceVisualizer.h/.cpp</code>	소스 위치를 뷰포트에 박스/화살표로 렌더링
<code>GammaTonRayVisualizer.h/.cpp</code>	<code>Trace Single Ray</code> 결과 경로를 뷰포트에 선분으로 렌더링

4. 핵심 데이터 구조

모든 타입은 `GTCore.h`에 정의되어 있습니다.

GTSurfel — 표면 원소 (삼각형 하나 = surfel 하나)

```
struct GTSurfel {
    GTVec3      position;           // 삼각형 중심 좌표
    GTVec3      normal;            // 삼각형 법선
    GTVec2      uv;                // 아틀라스 UV (텍스처 쓰기 위치)
    int         geom_id;           // 어느 메시(액터)에 속하는지
    GTGammaReflectance base_reflectance; // 사용자 설정 감쇠율 (불변)
    GTGammaReflectance reflectance;    // 현재 유효 감쇠율 (매 iter 갱신)
    GTMaterialProps material;        // 이 표면에 누적된 재질 (물리 피드백용)
};
```

`material`은 두 가지 용도로 쓰입니다:

- **물리 피드백**: `sh_surf`가 높으면 다음 γ -ton의 `ks/kp`를 약화시킴 (젖은 표면)
- **PICKUP 규칙**: `surface.sp`를 γ -ton이 흡수 → stain-bleeding 구현

GTObjTexture — 에이징 텍스처 (픽셀 단위 결과)

```
struct GTObjTexture {
    std::vector<float> sd; // R 채널: 먼지 밀도
    std::vector<float> sp; // G 채널: 색소 (녹, 이끼)
    std::vector<float> sr; // B 채널: 거칠기
    std::vector<float> sh; // A 채널: 습도
};
```

`GTSurfel.material`과 `GTObjTexture`는 같은 정보를 두 곳에 저장합니다.

- `material` → 물리 시뮬레이션 피드백 (surfel 단위)
- `GTObjTexture` → 시각 출력 (픽셀 단위, 최종 텍스처)

두 곳이 동기화되는 시점: `traceTon()`의 `settle` 이벤트 + `applyCrossChannel()`.

GTTon — 비행 중인 γ -ton

```
struct GTTon {
    GTMotionProbs  motion;    // ks, kp, kf (합 ≤ 1, 나머지가 settle 확률)
    GTMaterialProps carrier;  // 들고 다니는 재질 sd/sp/sr/sh
};
```

GTSimConfig — 전체 설정 묶음

```
struct GTSimConfig {
    int    n_tons_per_iter;    // 매 iter당 발사 수
    int    max_bounces;        // 최대 반사 횟수
    float  flow_step;          // kf 이동 거리 (cm)
    float  deposit_k;          // 전역 침착 강도
    float  pickup_k;           // 전역 흡수 강도
    float  bounce_distance;     // kp 포물선 이동 거리 (cm)
    float  parabola_gravity;    // kp 중력 강도
    std::vector<GTTonType> ton_types; // 동시 발사  $\gamma$ -ton 종류들
};
```

5. Run 버튼 전체 실행 플로우

`SGammaTonPanel::OnRunClicked()` → `GTSimulator::runIteration()` × N iter 순서입니다.

```
OnRunClicked()
├── ① 선택된 Actor 목록 수집 (GEditor->GetSelectedActorIterator)
├── ② FGammaTonMeshBridge::BuildScene()
│   ├── 각 Actor의 UStaticMeshComponent에서 버텍스/UV/법선/BaseColor 추출
│   ├── GTMesh 배열 생성
│   ├── GTSurfel 배열 생성 (삼각형당 1개)
│   ├── GTObjTexture 배열 초기화 (액터당 1개)
│   └── GTRayIntersector.addMesh() + commit() → Embree BVH 빌드
├── ③ 기존 텍스처 프리로드 (이전 실행 결과 누적을 위해)
│   └── /Game/GammaTon/{ActorName}_Dust 에셋이 있으면 GTObjTexture에 로드
├── ④ GTSimConfig 구성 (UI 파라미터 → GTSimConfig)
├── ⑤ GTSimulator 생성
├── ⑥ 이터레이션 루프 (진행 다이얼로그 포함)
│   └── for (int iter = 0; iter < NumIterations; iter++) {
│       └── sim.runIteration() ← n_tons_per_iter 개  $\gamma$ -ton 추적
│   }
```

```

        sim.applyCrossChannel()    ← sh→sr 녹 성장, sh 증발 등
        sim.updateReflectance()    ← surfel의 sr/sh → delta_s/p/f 갱신
    }
    for (GTObjTexture& tex : textures)
        tex.blur(2)                ← 5×5 Gaussian blur 2회 (샷 노이즈 제거)
}
⑦ FGammaTonTextureBridge::CreateAndSaveTexture()
    GTObjTexture → UTexture2D → /Game/GammaTon/{Name}_Dust 에셋 저장
}
⑧ FGammaTonTextureBridge::ApplyToComponent()
    저장된 UTexture2D를 머티리얼 MID에 주입, DustColor/PigmentColor 설정

```

6. γ-ton 1개 트레이싱 코드 경로

`GTSimulator::traceTon()` 내부 바운스 루프입니다.

```

traceTon(ton, type, stats, rng)
|
|─ 소스 선택 및 origin/dir 샘플링 (sampleSource)
|
|─ for (int bounce = 0; bounce < max_bounces; bounce++)
|   |
|   |─ intersector_.intersect(origin, dir) ← Embree 레이캐스트
|   |   miss 시: stats.escaped++ → return
|   |
|   |─ [deteriorate] ton.motion.deteriorate(surfel.reflectance)
|   |   ks -= delta_s, kp -= delta_p, kf += kp잔량 - delta_f
|   |
|   |─ [sh 피드백] 젖은 표면이면 ks, kp 약화
|   |   ks_eff = ks × (1 - 0.5 × sh_surf)
|   |   kp_eff = kp × (1 - 0.3 × sh_surf)
|   |
|   |─ [BaseColor 피드백] 표면 밝기 → ton.carrier.sp에 pickup_k만큼 추가
|   |   어두운 틈새에서 sp가 더 빨리 쌓이는 효과
|   |
|   |─ [PICKUP 규칙] 사용자 정의 규칙 적용
|   |   ex) SURFACE.sp → TON.sp × pickup_k (stain-bleeding 핵심)
|   |
|   |─ 난수 ξ 추출
|   |
|   |─ ξ < ks → [ks: 반구 반사]
|   |   carrier 감쇠 (sd×0.92, sp×0.95, sr×0.92, sh×0.88)
|   |   randomHemisphereDir(n) → 새 방향
|   |
|   |─ ξ < ks+kp → [kp: 포물선 바운스]
|   |   intersectParabolic() ← parabola_steps개 선분으로 중력 포물선 근사
|   |   다음 루프에서 landing point를 pending hit으로 처리
|   |
|   |─ ξ < ks+kp+kf → [kf: 표면 흐름]
|   |   FLOW 규칙으로 ton.sp/sh의 일부를 표면에 부분 침착

```

```

| 이동 방향: gravity_tangent×0.7 + random_tangent×0.3
| flow_step만큼 이동 후 -n 방향으로 재충돌 레이 발사
|
└─ 나머지 → [settle: 정착]
    w_sd = 0.2 + 0.8×upward (위쪽 면에 먼지 더 많이)
    w_sp = moisture_weight × (1 - upward×0.7) (sh≥0.2인 그늘에만 이끼)
    SETTLE 규칙으로 GTObjTexture.deposit() 호출 (3×3 Gaussian splat)
    surfel.material 동기 갱신
    stats.settled++ → return

```

포물선 근사 (`intersectParabolic`)

```

launch_dir (반구 샘플링)
|
├─ parabola_steps개 선분으로 분할
|   vel.z -= parabola_gravity/steps (매 스텝마다 중력 적용)
|
└─ 각 선분마다 Embree intersect → 첫 번째 히트 반환
    (디버그 모드에서는 waypoints 배열에 중간점 저장 → 뷰포트 호 렌더링)

```

7. 이터레이션 루프 상세

`runIteration()` 한 번이 "1세대 풍화"입니다.

```

runIteration()
|
├─ 가중치 합산 (GTTonType별 weight)
|
└─ for (int i = 0; i < n_tons_per_iter; i++)
    1. 가중치 기반 ton type 선택 (weighted random)
    2. GTTon 초기화 (type의 init_motion, init_carrier 복사)
    3. traceTon(ton, type, stats, rng)

```

각 이터레이션 후 순서:

```

runIteration() → 텍스처 + surfel.material에 재질 누적
applyCrossChannel() → 텍스처 + surfel 양쪽 동기 갱신
    tex.sr[i] += rust_rate × tex.sh[i] ← 시각 출력
    tex.sh[i] *= (1 - humidity_decay)
    surfel.material.sr += rust_rate × sh ← 물리 피드백
    surfel.material.sh *= (1 - humidity_decay)
updateReflectance() → surfel.reflectance 재계산
    delta_s_eff = base.delta_s + sr × 0.3 (거칠수록 γ-ton 잘 잡힘)

```

```
delta_p_eff = base.delta_p + sh × 0.2    (습할수록 흐름으로 전환 빠름)
delta_f_eff = base.delta_f + sh × 0.1
```

8. 텍스처 파이프라인

시뮬레이션 결과

```
GTObjTexture { sd[], sp[], sr[], sh[] } // float 배열 (픽셀 수 × 4)
```

```
tex.blur(2) - 5×5 Gaussian blur 2회
```

CreateAndSaveTexture()

BGRA8 엔리얼 텍스처로 변환

B=sr G=sp R=sd A=sh

/Game/GammaTon/{ActorName}_Dust.uasset 저장

ApplyToComponent()

원본 머티리얼을 복제 → MID(Material Instance Dynamic) 생성

MID 파라미터 주입:

AgingTexture ← 위에서 저장한 텍스처

DustColor ← UI 색상 피커

PigmentColor ← UI 색상 피커

DustTexture ← 선택적 디테일 텍스처

PigmentTexture ← 선택적 디테일 텍스처

AtlasUV 채널 선택 (라이트맵 UV가 있으면 채널 1 우선)

뷰포트에 풍화 결과 실시간 반영

RGBA 채널 매핑 요약:

채널	GTObjTexture 필드	UTexture2D	머티리얼 용도
R	sd	R	Lerp(OrigColor, DustColor×DustTex, sd) 알파
G	sp	G	Lerp(위 결과, PigColor×PigTex, sp) 알파
B	sr	B	Roughness += sr × RoughnessScale
A	sh	A	BaseColor × (1 - sh × WetnessScale)

9. 코드 수정 가이드

새로운 이벤트 종류 추가 (예: 흡착, 분리)

1. `GTCore.h`의 `GTTransportEvent` 열거형에 항목 추가
2. `GTSimulator::traceTon()`의 `else if` 체인에 새 이벤트 처리 추가
3. `SGammaTonPanel.h`의 `EventOptions_`와 `SGammaTonPanel.cpp`의 드롭다운 텍스트 목록에 추가

새로운 소스 타입 추가

1. `GTCore.h`의 `GTSourceType` 열거형에 항목 추가
2. `GT Simulator::sampleSource()`의 `switch`에 샘플링 로직 추가
3. `GammaTonSourceVisualizer`에 뷰포트 렌더링 추가 (선택)
4. `SGammaTonPanel.h`의 `SourceOptions_` 목록에 추가

물리 피드백 조건 변경 (예: sh 임계값 조정)

`GT Simulator.cpp`의 `traceTon()` 내 `settle` 분기:

```
const float kShThresh = 0.20f; // ← 이 값이 sp(이끼) 침착 임계값
float w_sp_moisture = (sh_surf < kShThresh)
    ? 0.0f
    : (sh_surf - kShThresh) / (1.0f - kShThresh);
```

새로운 Cross-Channel 규칙 추가

1. `GTCore.h`의 `GTCrossChannelRules` 구조체에 필드 추가
2. `GT Simulator::applyCrossChannel()`에 텍스처+surfel 양쪽 갱신 로직 추가
3. `SGammaTonPanel.h`에 UI 변수 추가, `SGammaTonPanel.cpp`의 `Construct()` UI 빌드에 슬라이더 추가

시뮬레이션을 UE 없이 단독 테스트하는 방법

`Core/` 폴더는 `#include <vector>, <cmath>` 등 STL만 사용합니다.

순수 C++ main 파일을 만들어 `GT Simulator`를 직접 구성하면 UE 없이 실행할 수 있습니다:

```
#include "GTCore.h"
#include "GTRayIntersect.h"
#include "GT Simulator.h"

int main() {
    GTMesh mesh = /* 메시 데이터 */;
    GTRayIntersector intersector;
    int geomId = intersector.addMesh(mesh);
    intersector.commit();

    auto surfels = GTGenerateSurfels(mesh, GTGammaReflectance{}, geomId);
    std::vector<GTObjTexture> textures = { GTObjTexture(512, 512) };

    GTSimConfig config;
    GT Simulator sim(surfels, intersector, {mesh}, config, &textures);
    sim.runIteration();
    // textures[0].sd[], sp[] 등을 PNG로 저장하여 확인
}
```

데이터 흐름 전체 요약


```

UE Actors (Static Mesh)
|
| MeshBridge::BuildScene()
▼
GTMesh[] + GTSurfel[] + GTObjTexture[] + GTRayIntersector (Embree BVH)
|
| GTSimulator::runIteration() × NumIterations
▼
GTObjTexture[] ← 재질 누적 (sd/sp/sr/sh 각 픽셀)
GTSurfel[].material ← 물리 피드백 누적
|
| (매 iter) applyCrossChannel() + updateReflectance()
|
| (최종) GTObjTexture.blur(2)
|
| TextureBridge::CreateAndSaveTexture()
▼
UTexture2D (/Game/GammaTon/{Name}_Dust)
|
| TextureBridge::ApplyToComponent()
▼
뷰포트에 풍화 결과 표시

```

논문: Chen et al., "Visual Simulation of Weathering by γ -ton Tracing", SIGGRAPH 2005